

MINDBOT

An NLP-Based Sentiment Analysis Mental Health Chatbot

Final Year Project — Complete Reference & Presentation Guide

Bilingual (English + Malay) · Emotion Classification · Mood Analytics

Prepared by: Muhammad Afiq

Matric No.: [your matric number]

Supervisor: [supervisor name]

Universiti Utara Malaysia

June 2026

Table of Contents

After opening this document in Word, right-click the table below and choose “Update Field” to populate page numbers.

Table of Contents	2
1. Executive Summary	4
Project at a glance	4
2. Problem Statement & Objectives	4
Objectives	4
3. System Overview & Logic Flow	5
End-to-end flow of a single message	5
4. Dataset & Preprocessing	5
Preprocessing steps.....	5
5. Model Training Methodology.....	6
Why DistilBERT (multilingual)?	6
Training configuration.....	6
6. The Hybrid Classification Pipeline	6
Layer 1 — Small-talk shortcut	6
Layer 2 — Crisis safety net (highest priority).....	6
Layer 3 — ML classifier (primary engine).....	6
Layer 4 — Lexicon backstop	7
Layer 5 — Context carry-forward	7
7. Sentiment Scoring & Mood Analytics	7
The mood dashboard shows	7
8. Contextual Conversation	7
Context-aware understanding	7
Context-aware generation	7
9. Response Generation.....	7
Three-level graceful degradation.....	8
LLM backend (switchable).....	8
10. Backend Architecture.....	8
API endpoints.....	8
Database (SQLite) — three tables	8
11. Frontend	9
Three views.....	9
12. Safety & Ethical Design	9
13. Results & Evaluation.....	9

13.1 Classification performance	9
13.2 Baseline comparison — why deep learning?	10
13.3 Safety-layer evaluation	11
13.4 Error analysis	11
14. Technology Stack	11
15. Running the System (Demo Setup)	12
16. Suggested Demo Script	12
17. Presentation Tips & Tricks	13
Structure your talk	13
Slides	13
Delivery	13
18. Anticipated Questions & Answers	13
19. Limitations	14
20. Future Work	14
21. Conclusion	15
Appendix A — Project File Structure	16
Appendix B — Key Configuration	16

1. Executive Summary

Mindbot is a bilingual (English and Malay) mental-health support chatbot built around a Natural Language Processing (NLP) sentiment-analysis pipeline. For every user message it performs three things: (1) it classifies the underlying emotional state using a fine-tuned BERT model, (2) it generates an empathetic, context-aware reply, and (3) it records a continuous “wellbeing score” that is visualised as a personal mood dashboard.

The project demonstrates the full machine-learning lifecycle — data preparation, model fine-tuning, quantitative evaluation against baselines, and deployment inside a working full-stack web application — with particular attention to the safety requirements of the mental-health domain.

Project at a glance

Aspect	Detail
Title	NLP-Based Sentiment Analysis Mental Health Chatbot (Mindbot)
Core task	4-class emotion classification: Anxiety, Depression, Suicidal, Normal
Languages	English and Malay (bilingual)
Model	Fine-tuned DistilBERT (distilbert-base-multilingual-cased)
Accuracy	79.1% overall (English 80.0%, Malay 78.3%), macro-F1 0.79
Stack	Flask + SQLite backend, vanilla JS + Chart.js frontend
LLM	Claude API or local Ollama for response generation (switchable)

2. Problem Statement & Objectives

Mental-health support is not always accessible — stigma, cost, and a shortage of professionals mean many people, especially students, have no safe outlet to express how they feel. Most existing chatbot solutions are English-only and treat every message in isolation, ignoring the emotional thread of a conversation.

Objectives

1. To build an NLP model that accurately classifies the emotional state behind a user’s message in both English and Malay.
2. To convert the model’s output into a continuous sentiment score and track a user’s mood over time.
3. To generate empathetic, context-aware responses that maintain the conversation’s emotional thread.
4. To incorporate a reliable safety mechanism for detecting and responding to crisis (self-harm) language.
5. To deliver all of the above inside a usable full-stack web application.

3. System Overview & Logic Flow

Mindbot follows a three-stage pipeline: Classify → Generate → Track. The NLP classifier runs on every message (this is the sentiment-analysis core), its output steers the response generator, and the result is persisted so the analytics layer can chart mood trends.

End-to-end flow of a single message

1. The user sends a message from the browser (frontend) to the Flask /chat endpoint, with a JWT token for authentication.
2. The backend looks up the user's previous emotion (for conversational context).
3. `classify_text()` runs the hybrid classifier and returns an emotion, confidence, crisis flag, and full probability distribution.
4. `score_from_distribution()` converts the distribution into a wellbeing score between -1.0 and +1.0.
5. `generate_response()` produces an empathetic reply — a context-aware LLM generation, or a template from the curated bank if the LLM is offline.
6. Both the user and bot messages (with the emotion and sentiment score) are saved to the SQLite database.
7. The JSON response is returned and rendered, with an emotion chip shown on the user's message.

4. Dataset & Preprocessing

A balanced bilingual dataset of 4,400 examples was assembled from two sources. The English data was sampled from a public mental-health dataset; the Malay data was machine-translated from the same English source to create a parallel corpus.

Language	Source file	Examples
English	mental_heath_unbalanced.csv	600 per class (2,400)
Malay	malay_dataset_translated.csv	500 per class (2,000)
Total	—	4,400 (balanced across 4 classes)

Preprocessing steps

- Filtered to the four target classes (Anxiety, Depression, Suicidal, Normal) and dropped rows with missing text.
- Balanced sampling (equal examples per class) to prevent the model from favouring the majority class.
- Stratified 80/10/10 split into train (3,520), validation (440) and test (440) — stratification keeps the class ratio identical in each split.
- Tokenisation with the multilingual WordPiece tokenizer, truncated/padded to 128 tokens.

5. Model Training Methodology

Why DistilBERT (multilingual)?

- **Multilingual:** pre-trained on 104 languages including Malay, so it already understands Malay subword tokens.
- **Distilled:** about 40% smaller and ~60% faster than full BERT while retaining ~97% of its language understanding — essential for training and inference on a CPU-only machine.
- **Classification-ready:** designed to output a probability distribution over classes, which we need for the wellbeing score.

Training configuration

Hyperparameter	Value
Base model	distilbert-base-multilingual-cased
Epochs	3
Batch size	16 (train) / 32 (eval)
Max sequence length	128 tokens
Warmup steps / weight decay	100 / 0.01
Model selection	Best checkpoint by weighted F1 on validation set
Hardware	CPU (no GPU required)

Training is reproducible via `train_bilingual.py`, which also writes a per-language test report (`bilingual_report.txt`).

6. The Hybrid Classification Pipeline

A pure machine-learning model is ~80% accurate, but in a mental-health context a missed distress signal is dangerous. Mindbot therefore wraps the ML model in a transparent, auditable rule layer. This hybrid design is a key talking point for the defense.

Layer 1 — Small-talk shortcut

Greetings and farewells (“hi”, “bye”, “selamat pagi”) are matched directly and skip the model, avoiding noisy predictions on trivial input.

Layer 2 — Crisis safety net (highest priority)

A deterministic list of explicit self-harm phrases (English and Malay) is matched first. If any is present, the message is force-flagged as SUICIDAL regardless of what the model predicts. For self-harm we deliberately prefer to over-trigger rather than ever miss.

Layer 3 — ML classifier (primary engine)

DistilBERT produces a probability distribution over the four classes; the top class becomes the predicted emotion and its probability the confidence.

Layer 4 — Lexicon backstop

If the model predicts NORMAL but a clear distress keyword is present, the lexicon corrects this false-negative (the dangerous direction). The model does the heavy lifting; the lexicon only rescues the edges.

Layer 5 — Context carry-forward

Short follow-ups such as “yeah” or “it’s my exams” would look NORMAL on their own. If the previous turn was distressed and the user has not signalled improvement, the prior emotion is carried forward (a suicidal disclosure decays to sustained concern rather than repeatedly re-firing crisis resources).

7. Sentiment Scoring & Mood Analytics

Instead of discarding the probability distribution and keeping only the top label, Mindbot collapses the whole distribution into a single continuous wellbeing score:

$\text{score} = \sum P(\text{class}) \times \text{valence}(\text{class})$ range: -1.0 (crisis) ... +1.0 (positive)

Each class has a valence weight (Normal +1.0, Anxiety -0.45, Depression -0.7, Suicidal -1.0, etc.). This score is stored on every user message and powers the dashboard.

The mood dashboard shows

- Average mood and a human-readable band (Positive / Stable / Low / Distressed / Crisis).
- Recent trend (last 5 messages vs the previous 5) — improving, steady, or declining.
- A sentiment-over-time line chart and an emotion-distribution doughnut chart.
- A count of high-risk (crisis) flags.

8. Contextual Conversation

Mindbot is contextual in two complementary ways:

Context-aware understanding

The classifier carries emotional state across short follow-ups (Layer 5 above), so the bot does not lose the thread when the user replies briefly. It resets to NORMAL when the user clearly signals they feel better (“actually I feel better now, thanks”).

Context-aware generation

Rather than dressing up a fixed template, the language model generates a fresh reply grounded in the whole recent conversation (last six turns) and steered by the detected emotion. This lets the bot say things like “given your deadline is next week...” instead of a generic response.

9. Response Generation

Replies are produced in the “Solace” persona — warm, validating, non-judgmental, never offering diagnosis. The persona prompt also enforces an absolute language rule so a Malay user always receives a Malay reply.

Three-level graceful degradation

1. Preferred: the configured LLM generates a context-aware reply.
2. Fallback: if the LLM is unavailable, a curated template for the detected emotion is rephrased.
3. Last resort: if no LLM at all is reachable, the raw template is returned — so the user always gets a reply (important for a live demo).

LLM backend (switchable)

- **Claude API** (claude-haiku-4-5) — fast and low-cost, needs an API key.
- **Ollama** (llama3.2) — fully local and free, no API key required.

The backend is selected via the LLM_PROVIDER setting; crisis responses are always deterministic (fixed hotline numbers) and never left to the LLM.

10. Backend Architecture

The backend is a Flask application organised into blueprints, using JWT tokens for authentication and bcrypt for password hashing. All chat and analytics endpoints require a valid token.

API endpoints

Method & Route	Purpose
POST /auth/register	Create an account, return a JWT
POST /auth/login	Authenticate, return a JWT
POST /chat	Classify a message, generate + persist a reply
GET /conversations	List the user’s conversations
GET /conversations/<id>	Fetch one conversation with messages
DELETE /conversations/<id>	Delete a conversation
GET /analytics/summary	Headline mood stats
GET /analytics/mood	Daily average sentiment time series
GET /analytics/emotions	Emotion distribution counts

Database (SQLite) — three tables

- **User:** id, name, email, password hash.
- **Conversation:** belongs to a user; has a title and timestamps.
- **Message:** role (user/bot), content, and — for user messages — the emotion, confidence, sentiment score and crisis flag.

11. Frontend

The frontend is a single-page application (SPA) written in vanilla JavaScript with Chart.js for visualisation — no build step, which keeps it simple to run and explain.

Three views

- **Authentication:** register / login; the JWT is stored in the browser.
- **Chat:** conversation sidebar (create / load / delete), message bubbles, colour-coded emotion chips, and a typing indicator.
- **Mood Dashboard:** stat cards plus the sentiment-over-time and emotion-distribution charts.

12. Safety & Ethical Design

Because the domain is sensitive, safety is built into the architecture rather than bolted on.

Risk	Mitigation
Model misses self-harm language	Deterministic crisis safety net (always overrides the model)
Confident-but-wrong NORMAL on distress	Lexicon backstop corrects false negatives
User in crisis	Fixed response with Talian Kasih 15999 and Befrienders KL 03-7627 2929
Losing emotional context	Context carry-forward in the classifier
LLM unavailable mid-demo	Three-level template fallback
Mistaken for a professional	AI disclaimer shown on the first message

13. Results & Evaluation

All evaluation uses the same held-out test set (440 examples the model never saw during training), so the numbers are honest and the comparisons are fair.

13.1 Classification performance

Test set	Accuracy	Macro-F1
Overall	79.1%	0.79
English only	80.0%	0.80
Malay only	78.3%	0.78

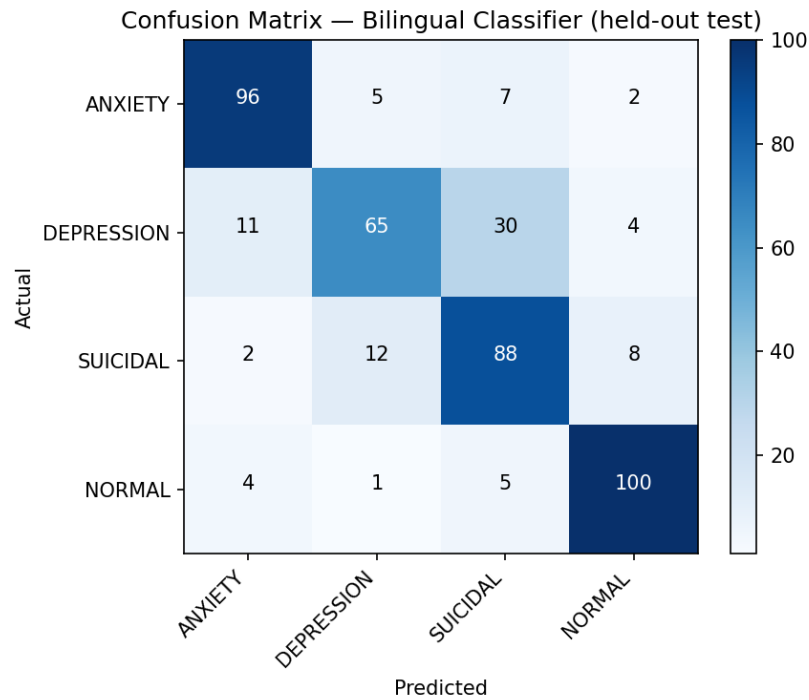


Figure 1. Confusion matrix on the held-out test set. The strong diagonal shows correct predictions; the main off-diagonal is Depression vs Suicidal.

13.2 Baseline comparison — why deep learning?

To justify using DistilBERT, two classical machine-learning baselines were trained on the same data and tested on the same set.

Model	Accuracy	Macro-F1
TF-IDF + Logistic Regression	0.69	0.68
TF-IDF + Linear SVM	0.70	0.70
DistilBERT (ours)	0.79	0.79

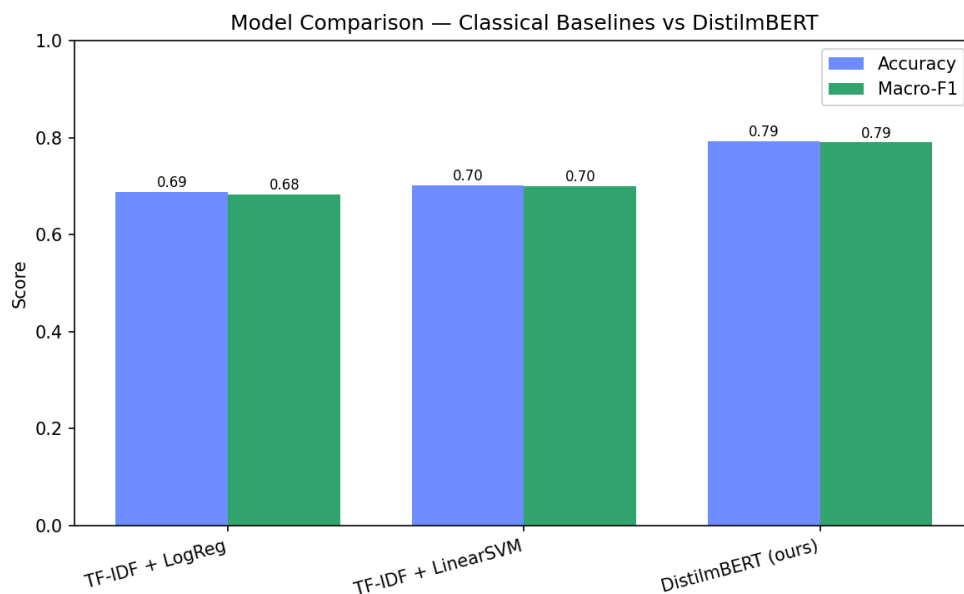


Figure 2. DistilBERT outperforms the best classical baseline by ~9 percentage points, justifying the deep-learning approach.

13.3 Safety-layer evaluation

Crisis detection was evaluated as a binary problem (crisis vs not). Recall matters most here — a missed crisis is far worse than a false alarm.

Metric	Result
Crisis recall (caught)	80.9%
Precision	67.4%
Specificity	87.0%
Explicit crisis phrases caught	14 / 14
False alarms on safe text	0 / 6

13.4 Error analysis

The dominant error is Depression being misread as Suicidal (and vice-versa). These states share a vocabulary of hopelessness and worthlessness, so the boundary is genuinely fuzzy even for humans — which is exactly why the deterministic crisis safety net exists instead of trusting the classifier alone.

14. Technology Stack

Layer	Technologies
NLP / ML	Hugging Face Transformers, PyTorch, scikit-learn (baselines & metrics)

Layer	Technologies
Backend	Flask, Flask-JWT-Extended, Flask-SQLAlchemy, Flask-Bcrypt, SQLite
LLM	Anthropic Claude API (claude-haiku-4-5) or Ollama (llama3.2)
Frontend	HTML, CSS, vanilla JavaScript, Chart.js
Tooling	matplotlib, pandas (evaluation artefacts)

15. Running the System (Demo Setup)

Two terminals — one for the backend API, one to serve the frontend:

```
# Terminal 1 – backend
cd backend
python app.py                # serves http://127.0.0.1:5000
# Terminal 2 – frontend (from project root)
python -m http.server 8080 --directory frontend
# then open http://127.0.0.1:8080
```

Pre-demo checklist:

- Start the backend a few minutes early — the model loads into memory on first run.
- Send one warm-up message so the first (slow) LLM call happens before you present.
- If using Ollama, confirm it is running (ollama serve) and llama3.2 is pulled.
- Have the eval_output charts open in a tab as backup in case of any live issue.

16. Suggested Demo Script

A short, rehearsed path that shows every feature in under three minutes:

1. Register a new account (shows auth + database).
2. Send: “I’ve been feeling really anxious about my final year project.” — point out the ANXIETY chip and confidence.
3. Reply: “yeah I can’t sleep because of it” — show context carry-forward (still distress, not NORMAL).
4. Reply: “I feel a bit better after talking, thanks” — show recovery detection (resets to NORMAL).
5. Type a Malay message (e.g. “saya rasa sangat sedih”) — show bilingual classification and a Malay reply.
6. Open the Mood Dashboard — show the sentiment arc and emotion chart.
7. (Optional, with care) Show a crisis message triggering the fixed hotline response, to demonstrate the safety net.

Tip: prepare the exact sentences in advance — do not improvise wording during the live demo.

17. Presentation Tips & Tricks

Structure your talk

- Open with the problem (accessibility of mental-health support), not the technology.
- State your contribution in one sentence: a bilingual sentiment-analysis pipeline with safety-first crisis handling and mood tracking.
- Spend the most time on the parts you built and understand deeply: the dataset, the model, and the hybrid pipeline.
- End with limitations and future work — it shows maturity and pre-empts the panel's questions.

Slides

- Use the confusion matrix and the baseline bar chart — examiners trust visuals with numbers.
- One architecture diagram (Classify → Generate → Track) is worth several paragraphs.
- Keep code off the slides; show at most one short snippet (the wellbeing-score formula works well).

Delivery

- Always have a backup: a screen-recording of the demo and the saved charts, in case the live demo fails.
- Know your numbers cold: 79% accuracy, 0.79 macro-F1, ~9-point gain over the baseline, ~81% crisis recall.
- Be honest about limitations — “I know X is a weakness, and here is how I would address it” scores better than pretending it is perfect.
- Frame the LLM correctly: the sentiment-analysis NLP is your contribution; the LLM only phrases the reply.

18. Anticipated Questions & Answers

Q: Why not just use ChatGPT / an LLM for everything?

A: The core contribution is the sentiment-analysis pipeline — classifying emotion, producing a continuous wellbeing score, and persisting it for analytics. A generative model has no concept of tracking a user's mood over time, and crisis detection must be deterministic for safety, not left to a generative model.

Q: What is your model's accuracy and how did you measure it?

A: 79.1% overall (80% English, 78% Malay) and 0.79 macro-F1 on a held-out test set of 440 examples the model never saw. I also report a confusion matrix and per-class precision/recall/F1.

Q: Why DistilBERT and not full BERT or a larger model?

A: DistilBERT keeps ~97% of BERT's accuracy while being ~40% smaller and faster, which let me train and run on a CPU. My baseline comparison shows it already beats classical models by ~9 points; a larger model adds cost for marginal gain at this dataset size.

Q: Why not classical ML like SVM — it is simpler?

A: I tested exactly that. TF-IDF + Logistic Regression scored 0.68 and TF-IDF + Linear SVM 0.70 macro-F1, versus 0.79 for DistilBERT on the same split — a clear, measured justification for deep learning.

Q: How do you handle Malay?

A: Three ways: the model is multilingual and trained on both languages; a language detector routes the reply language; and the lexicon backstop covers distress keywords in both languages.

Q: How does the wellbeing score work?

A: It is a weighted sum of the full class-probability distribution, where each emotion has a valence from +1 (Normal) to -1 (Suicidal). This turns a single label into a continuous value that can be averaged and charted over time.

Q: How do you ensure safety for suicidal users?

A: A deterministic crisis safety net matches explicit self-harm phrases and always overrides the model, returning fixed Malaysian helpline numbers. It catches 14/14 explicit test phrases, and overall crisis recall is ~81%. Crisis replies are never generated by the LLM.

Q: What are the limitations?

A: The Malay data is machine-translated rather than professionally written; the model does not yet handle heavy code-switching (Manglish); the context window is six turns; and four classes cannot capture every nuance. Each has a concrete future-work answer.

Q: Is the chatbot a replacement for therapy?

A: No, and the app states this explicitly with a disclaimer. It is a supportive, first-line tool that encourages professional help and surfaces crisis resources.

Q: How is user data protected?

A: Passwords are bcrypt-hashed, sessions use signed JWT tokens, and each user can only access their own conversations. Data is stored locally in SQLite for the demo.

19. Limitations

- Malay training data is machine-translated, which can introduce unnatural phrasing; professional translation would improve Malay accuracy.
- Code-switching (mixed Malay-English / “Manglish”) is not explicitly modelled.
- Conversational memory is limited to the last six turns; very long sessions lose earlier context.
- Four emotion classes simplify a nuanced emotional space.
- Depression and Suicidal remain the hardest pair to separate, which is mitigated but not eliminated by the safety net.

20. Future Work

- Collect or commission a professionally written Malay mental-health dataset, including Manglish.
- Add a rolling conversation summary so context persists across long sessions.

- Expand the emotion taxonomy (e.g. stress, anger, grief) and add intensity levels.
- Add clinician-style reporting / exportable mood history for users to share with a counsellor.
- Deploy to a secure server with HTTPS and proper secret management for real-world use.

21. Conclusion

Mindbot delivers a complete, defensible NLP project: a fine-tuned bilingual sentiment classifier (79% accuracy, clearly beating classical baselines), a continuous wellbeing score that powers mood analytics, context-aware empathetic responses, and a safety-first design for the crisis case — all inside a working full-stack application. The combination of measured evaluation and practical engineering makes it both academically sound and genuinely useful.

Appendix A — Project File Structure

backend/	
app.py	Flask app factory + entry point
config.py	Central configuration
database.py	SQLAlchemy models (User/Conversation/Message)
auth.py	Register / login / JWT
routes_chat.py	/chat + conversation endpoints
routes_analytics.py	Mood analytics endpoints
model.py	Hybrid classifier (ML + safety net + context)
sentiment.py	Distribution -> wellbeing score
llm_rephraser.py	LLM response generation + language detection
response_generator.py	Routing + template fallback
evaluate.py	Confusion matrix + metrics
baseline_comparison.py	Classical ML vs DistilBERT
safety_eval.py	Crisis-detection metrics
error_analysis.py	Qualitative error breakdown
data_utils.py	Shared dataset loading/splitting
local_model/	The fine-tuned model
train_bilingual.py	Training script (project root)
frontend/	index.html, script.js, style.css
data/	datasets + SQLite database

Appendix B — Key Configuration

All values are environment-overridable (see backend/config.py):

Setting	Default / Meaning
LLM_PROVIDER	“ollama” (local) or “claude” (API)
ANTHROPIC_MODEL	claude-haiku-4-5
OLLAMA_MODEL	llama3.2
DATABASE_URL	SQLite file in /data
JWT_SECRET_KEY / SECRET_KEY	Change for any real deployment
HISTORY_TURNS	6 (turns of context fed to the LLM)

— End of document —